Ganita Manjari | Grade 9 — Mathematics

Chapter 11: Algorithms & Data Structures • Section 11.3

KEY CONCEPT: WHAT IS A DATA STRUCTURE?

When executing an algorithm, intermediate quantities (like a list of divisors) are stored using names. A **list** arranged in ascending order is an example of a **data structure**. Data structures allow us to organize and manage information efficiently.

Exercise Set 11.2

Exercise Questions

1. Suppose **List 1** and **List 2** are two lists of numbers arranged in increasing order:

   (i) Write an algorithm to find elements in **List 1** that are *not present* in **List 2**.

   (ii) Write an algorithm to find elements in **List 2** that are *not present* in **List 1**.

2. Describe an algorithm to compute the **least common multiple (LCM)** of two numbers.

*3. Divisors occur in pairs. For instance, the divisors of 18 are **(1, 18)**, **(2, 9)**, and **(3, 6)**.

   (i) If we write out divisors in pairs, how many numbers do we have to examine between 1 and *n* to find all the divisors of *n*?

   (ii) If we list out the divisors in pairs, will our GCD algorithm still work in the manner we have described?

Simplified Concept & Solution Hints

Simplified Thinking Strategy

For Q1: Since both lists are sorted in increasing order, you don't need to re-scan from the start each time. Compare elements step-by-step from left to right!

For Q2: Remember the identity: `LCM(a, b) = (a × b) / GCD(a, b)`, or loop through multiples of the larger number.

For Q3: Pairs mean if d divides n, then n/d is also a divisor. You only need to check numbers up to √n!

11.3.1 Improving the Algorithm (Reference Material)

To make algorithms faster, we look for ways to reduce the number of checks or steps required to find common divisors and GCDs.

Algorithm Improvement Strategies

Method	How it Works	Efficiency Gain
Separate Scans	Find divisors of *m* (1 to *m*), then find divisors of *n* (1 to *n*).	Requires `m + n` total checks.
Combined Scan	Scan from 1 to `max(m, n)` and add to respective divisor lists simultaneously.	Reduces checks to `max(m, n)`.
Direct Common Divisors	Scan from 1 to `min(m, n)` and check if `j` divides both *m* and *n*.	Reduces checks further to `min(m, n)`.
Do Away With Lists	Track only the *largest* common divisor found so far, replacing older values.	Saves memory (no need to store full lists).

KEY TAKEAWAY FOR STUDENTS

An efficient algorithm not only gives the correct answer, but also minimizes the **number of steps** (time efficiency) and **memory used** (space efficiency).